

Sparse Knowledge Graphs, Joint Embeddings, and Multimodal Alignment:

A Linear-Algebraic Framework for Structured World Models

Niklaus Parcell
Mont Ops Inc. / Independent Research
nik@mont-ops.com

May 4, 2026

Abstract

We present a mathematical framework for representing structured world knowledge as a *sparse* relational system and coupling it to *dense* embedding geometries suitable for multimodal perception (vision, language, tabular signals, and robotics sensing). The abstraction separates (i) the *combinatorial* object—a knowledge graph with a small support relative to a full Cartesian product space—from (ii) *learned* or *hand-specified* vector representations and scoring maps used for similarity, retrieval, control, and transparency-oriented analysis. The formulation is implementation-agnostic but is motivated by high-performance numerical stacks that expose sparse–dense linear algebra and batched inner-product kernels (e.g., CSR/COO structures paired with dense embedding tables). We relate the graph–embedding interface to standard knowledge graph embedding objectives and sketch extensions to shared latent spaces for heterogeneous modalities without committing to a specific training regime or dataset scale. A companion note [1] specifies how the dense parameter bundle $(\mathbf{E}, \Theta)$ is allocated, initialized, fitted by optimization, and persisted—rather than assumed given *a priori*.

1 Introduction

Modern mechanical and embodied systems integrate heterogeneous sensors (cameras, ranging, inertial measurements, force/torque, language interfaces) with latent representations inherited from language-only machine learning. A recurring design tension is that *predictive* models do not necessarily encode *physical* constraints, compositional structure, or auditable relational structure in a form that downstream planners, safety monitors, or human operators can inspect.

Knowledge graphs (KGs) offer an explicit handle on *entities* and *relations*, but naively materializing all conceivable relational assertions as dense structures is infeasible: edges are sparse in practice, and the space of hypotheses grows combinatorially in the product of entity and relation inventories.

We adopt a deliberately **two-level** view:

1. A **sparse structural layer** $\mathcal{S}$ that records which relational facts, constraints, or logical scaffolds are asserted.
2. A **geometric layer** $\mathcal{E}$ of embeddings and smooth (or piecewise-smooth) maps that support similarity, scoring, composition with sensory encoders, and—when desired—attribution or influence analysis along structured coordinates.

This note makes the interfaces between $\mathcal{S}$ and $\mathcal{E}$ explicit in linear-algebraic form suitable for reproducible software implementations.

Scope

We do not claim new experimental benchmarks. We provide definitions, constructions sufficient for implementation, and a notational bridge between sparse graph operators and multimodal joint embeddings. We intentionally allow both *classical* KG scoring functions (distance, bilinear, complex/rotation models) and *hybrid* models where edges carry weights derived from streaming statistics or interaction operators.

2 Preliminaries and notation

Let $[n] := \{1, \dots, n\}$. Scalars are Roman or Greek; vectors are bold lower case (e.g., $\mathbf{v}$); matrices are bold upper case (e.g., $\mathbf{A}$). The transpose of $\mathbf{A}$ is $\mathbf{A}^\top$. For a finite set X , $|X|$ denotes cardinality.

3 A knowledge graph as a sparse relational structure

3.1 Entities, relations, and facts

Definition 1 (Knowledge graph signature). *A signature is a tuple $\Sigma = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ where $\mathcal{E}$ is a finite set of entities, $\mathcal{R}$ a finite set of relation types, and $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ a set of asserted facts (directed labeled edges).*

We enumerate entities and relations by fixed bijections

$$\text{id}_{\mathcal{E}} : \mathcal{E} \rightarrow [N], \quad \text{id}_{\mathcal{R}} : \mathcal{R} \rightarrow [L].$$

A fact $(e_h, r, e_t) \in \mathcal{T}$ is equivalently encoded as a triple of indices $(h, r, t) \in [N] \times [L] \times [N]$.

Remark 1 (Sparsity as structural assumption). *The full product $[N] \times [L] \times [N]$ has cardinality N^2L . In applications of interest, $|\mathcal{T}| \ll N^2L$. Algorithms should depend on $|\mathcal{T}|$ and the sparsity pattern of derived operators, not on materializing the dense hypothesis space.*

3.2 Sparse matrix encodings

Multiple matrix encodings induce the same graph; the correct choice depends on the operator one wishes to apply (propagation along entities, incidence of triples, relation-specific slices, etc.).

Coordinate (COO) form. Store triples as index triples $(h_i, r_i, t_i)_{i=1}^M$ with optional weights $w_i \in \mathbb{R}$. This is the canonical “structural support” list.

Adjacency-style blocks. For each relation $\ell \in [L]$, one may define a sparse matrix $\mathbf{S}^{(\ell)} \in \mathbb{R}^{N \times N}$ with

$$S_{ab}^{(\ell)} = \sum_{\substack{(h,r,t) \in \mathcal{T} \\ h=a, r=\ell, t=b}} w_{h,r,t}, \quad (1)$$

with nonnegative weights $w_{h,r,t}$ defaulting identically to one when unweighted facts are modeled.

Bipartite incidence (optional). Let $\mathcal{T} = \{\tau_1, \dots, \tau_M\}$ enumerate facts. An incidence $\mathbf{B} \in \mathbb{R}^{N \times M}$ between entities and facts can encode head endpoints (and a separate matrix for tails), enabling structured message passing distinct from squaring the entity space.

Definition 2 (Sparse structural operator space). *Fix a family of linear maps $\{\mathbf{L}_k\}_{k=1}^K$ derived from $\mathcal{T}$ (e.g., normalized adjacencies, incidence products, relation concatenations, or learned structural filters). The structural layer is the span of operations implementable as sparse–dense compositions $\mathbf{L}_k \mathbf{X}$ for dense $\mathbf{X}$ stored columnwise.*

This is the precise sense in which “sparse matrices represent the knowledge graph”: the graph determines sparse linear operators, not necessarily an embedding by itself.

Proposition 1 (Support of relation slices). *Let $\mathcal{T}$ be encoded by a list of M triples (h_i, r_i, t_i) with associated nonnegative weights $(w_i)_{i=1}^M$. For each $\ell \in [L]$, form $\mathbf{S}^{(\ell)}$ as in (1) by aggregating all list entries with $r_i = \ell$. Then each triple contributes mass only to entry $S_{h_i, t_i}^{(r_i)}$, and hence*

$$\sum_{\ell=1}^L \text{nnz}(\mathbf{S}^{(\ell)}) \leq M_{\text{dist}}, \quad (2)$$

where $\text{nnz}(\cdot)$ counts nonzero entries after aggregation and M_{dist} is the number of distinct index triples (h_i, r_i, t_i) in the list (so $M_{\text{dist}} \leq M$). In particular, if all triple keys are distinct, $\sum_{\ell} \text{nnz}(\mathbf{S}^{(\ell)}) \leq M$.

Proof sketch. Initialize empty sparse accumulators for each $\mathbf{S}^{(\ell)}$. Processing triple i performs only an update to coordinate (h_i, t_i) of $\mathbf{S}^{(r_i)}$, hence touches at most one accumulator cell keyed by (h_i, r_i, t_i) before merging. After aggregation, each nonzero in $\mathbf{S}^{(\ell)}$ arises from at least one distinct triple key with relation label ℓ ; distinct keys correspond to distinct merged positions, yielding the bound. $\square$

Proposition 2 (Complexity of one sparse–dense propagation). *Let $\mathbf{L} \in \mathbb{R}^{N \times N}$ be sparse with $Z = \text{nnz}(\mathbf{L})$ and let $\mathbf{X} \in \mathbb{R}^{N \times d}$ be dense. The matrix product $\mathbf{Y} = \mathbf{L}\mathbf{X}$ can be computed in $O(Zd)$ scalar multiply-adds using the standard compressed sparse row (CSR) algorithm, and dense storage for $\mathbf{Y}$ is Nd words.*

Proof sketch. CSR stores, for each row i of $\mathbf{L}$, the column indices j with $L_{ij} \neq 0$ and the corresponding values. Row i of $\mathbf{Y}$ satisfies $\mathbf{y}_i = \sum_{j: L_{ij} \neq 0} L_{ij} \mathbf{x}_j$, a linear combination of at most $\deg_{\text{row}}(i)$ rows of $\mathbf{X}$, each of length d . Summing $\deg_{\text{row}}(i)$ over i equals Z , hence $O(Zd)$ work; a column-oriented (CSC) scheme yields the same asymptotic bound. $\square$

Remark 2. Propositions 1–2 justify routines built from alternating sparse propagations $\mathbf{L}_k \mathbf{X}$ and dense batched linear maps: work scales linearly in the number of nonzero structural entries (bounded for relation slices by Proposition 1) and linearly in embedding width d , rather than in $N^2 L$.

4 Embedding geometry and scoring

4.1 Entity and relation parameters

Fix an embedding dimension $d \in \mathbb{N}$. A classical parametrization attaches to each entity $a \in [N]$ a vector $\mathbf{e}_a \in \mathbb{R}^d$ (row of a matrix $\mathbf{E} \in \mathbb{R}^{N \times d}$). Relation types may be represented by vectors, matrices, or complex tensors depending on the model class; we keep an abstract family $\Theta = \{\theta_r\}_{r=1}^L$.

4.2 Construction of the parameter bundle

In applications, $(\mathbf{E}, \Theta)$ are seldom provided externally as final artifacts. They are outputs of a *construction pipeline*: allocate tensors for (N, L, d) and the chosen relation-block schema; initialize (randomly, from a checkpoint, or with placeholders for engineering tests); optionally run stochastic optimization using batches of asserted triples, corrupted negatives, and a link-prediction loss; then persist for inference. Implementation layouts (row-major $\mathbf{E}$, structured storage for Θ vs. a single flattened vector) are orthogonal to the mathematical interface $s(h, r, t; \mathbf{E}, \Theta)$. See the companion note [1] for definitions, initialization modes, and a generic training loop.

4.3 Triple scoring abstractly

Definition 3 (Scoring map). *A scoring function is a map $s : [N] \times [L] \times [N] \rightarrow \mathbb{R}$ that measures compatibility of (h, r, t) under parameters $(\mathbf{E}, \Theta)$.*

Examples (sketch). Translation models induce scores from $\|e_h + \theta_r - e_t\|_p$. Bilinear models use $\langle e_h, \mathbf{W}_r e_t \rangle$ or complex inner products after identifying $\mathbb{R}^{2d}$ with $\mathbb{C}^d$. These examples are standard in KG embedding literature and serve here only to anchor notation; the framework applies to any s composed from inner products, distances, and smooth nonlinearities implemented in a numerical stack.

4.4 Regularities via structural operators

Many graph neural encoders and Transe-style objectives can be written as alternating applications of

$$\mathbf{H}^{(m+1)} = \sigma \left(\sum_k \mathbf{L}_k \mathbf{H}^{(m)} \mathbf{W}_k^{(m)} \right),$$

where $\mathbf{L}_k$ are sparse, $\mathbf{H}^{(m)}$ are dense node features, and σ is a nonlinearity. The present framework treats such recursions as *instances* of the structural operator family $\{\mathbf{L}_k\}$ acting on learned embeddings.

5 Multimodal encoders and a shared latent geometry

5.1 Modalities and encoder maps

Let $\mathcal{M}$ index modalities (e.g., text, image patches, tabular fields, sensor frames). For modality $m \in \mathcal{M}$, denote raw observations by ζ_m and encoder maps by

$$\psi_m : \mathcal{Z}_m \rightarrow \mathbb{R}^{d_m}, \tag{3}$$

which may be deep networks, classical signal processing front-ends, or identity maps on engineered features.

5.2 Projection into a joint space

Choose a joint dimension d . For each modality, define linear (or shallow) projections $\mathbf{W}_m \in \mathbb{R}^{d \times d_m}$ (or compatible nonlinear maps) and form joint vectors

$$\mathbf{z} = \sum_{m \in \mathcal{M}} \alpha_m \mathbf{W}_m \psi_m(\zeta_m) \quad \text{or} \quad \mathbf{z} = \text{concat}_m (\mathbf{W}_m \psi_m(\zeta_m)) \text{ after a fusion map.} \tag{4}$$

The correct fusion—sum, gated sum, attention, low-rank bilinear pooling—is task-dependent. The key mathematical commitment is that heterogeneous ζ_m become comparable only after explicit maps into a common Hilbert geometry where inner products define similarity.

5.3 Grounding visual or sensor tokens in lexical or entity geometry

Let $\mathcal{W}$ index text units (words, phrases, sentences) with vectors $\mathbf{u}_w \in \mathbb{R}^d$. An image patch or receptive field p embedded as $\mathbf{z}_p \in \mathbb{R}^d$ is *aligned* with language or KG semantics by maximizing (or targeting) similarity scores

$$\kappa(\mathbf{z}_p, \mathbf{u}_w) \quad \text{or} \quad \kappa(\mathbf{z}_p, \mathbf{e}_a), \quad (5)$$

for a chosen similarity κ (cosine, inner product after normalization, learned kernel).

This is the general formulation behind “regions map to phrases” and more broadly behind cross-modal retrieval once supervision or weak alignment is available.

6 Similarity, link prediction, and composite objectives

Training is intentionally left open at the level of hyperparameters; the companion note [1] gives a template loss-and-update loop acting on $(\mathbf{E}, \Theta)$. High-level objectives often combine:

- **Structure loss** $\mathcal{L}_{\text{KG}}(\mathbf{E}, \Theta; \mathcal{T})$ encouraging asserted triples to score above corrupted negatives.
- **Alignment loss** $\mathcal{L}_{\text{mm}}$ pulling paired cross-modal representations together (contrastive InfoNCE variants, kernel CCA-style objectives, or regression to graph targets).
- **Regularity on $\mathcal{S}$** enforcing sparsity patterns, rule penalties, or temporal decay on streaming edge weights when $\mathbf{S}^{(\ell)}$ is time-varying.

7 Action and output geometry

Let $\mathcal{A}$ denote an action or command space for an embodied agent (discrete or continuous). A map

$$\boldsymbol{\pi} : \mathbb{R}^d \rightarrow \mathcal{A} \quad \text{or} \quad \boldsymbol{\pi}_\theta : \mathbb{R}^d \rightarrow \Delta(\mathcal{A}) \quad (6)$$

from the joint latent to actions completes the interface from perception+KG alignment to behavior. When $\mathcal{A}$ is continuous and smooth, $\boldsymbol{\pi}$ may be differentiable end-to-end with respect to sensor encoders; when $\mathcal{A}$ is discrete, policy-gradient methods apply in the usual way.

Remark 3 (Separation of concerns). *The sparse KG layer supplies structured constraints and interpretable coordinates; the embedding layer supplies smooth optimization semantics; the action map supplies task specificity. This separation supports incremental software engineering (e.g., sparse operators in a dedicated linear algebra library with dense embedding blocks).*

8 Connection to interaction-based attribution (informal)

Attribution methods that accumulate pairwise influence between coordinates (for example via streaming updates to a symmetric interaction matrix with selectively zeroed diagonal) can be viewed as constructing a second family of sparse operators distinct from $\mathcal{T}$. When both coexist, a principled composition specifies whether interaction edges reference *features*, *entities*, or *latent*

factors, and how symmetries (same-modality vs cross-modality) rescale coupling. A rigorous integration is not developed here; the present note isolates the KG–embedding–multimodal interface required for physical AI stacks.

9 Discussion

The framework is deliberately modular: sparse relational structure, dense embeddings, multimodal projections, and control heads decouple cleanly in both software and mathematics. Implementation choices (CSR vs CSC, relation slicing vs unified incidence, `f32` vs `f64` numerical regimes) affect conditioning and throughput but not the abstract interfaces.

Future work beyond this note includes (i) analyzing identifiability of joint spaces under weak supervision, (ii) temporal and dynamic extensions $\mathcal{T}(t)$ with piecewise-smooth processes, (iii) tying graph-theoretic connectivity measures to controllability-style questions on the induced operators, and (iv) empirical calibration on sparse robotics telemetry coupled with language scaffolds.

References

- [1] N. Parcell. Constructing the Geometric Layer: Initialization, Parameter Blocks, and Optimization for Knowledge Graph Embeddings. Companion technical note `kg_embedding_construction.tex`, 2026.
- [2] A. Bordes, N. Usunier, A. García-Durán, J. Weston, O. Yakhnenko. Translating embeddings for modeling multi-relational data. In *NIPS*, 2013.
- [3] T. Trouillon, J. Welbl, S. Riedel, É. Gaussier, G. Bouchard. Complex embeddings for simple link prediction. *arXiv preprint* 1606.06357, 2016.
- [4] Z. Sun, J.-Y. Deng, J. Nie, J. Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. In *ICLR*, 2019.
- [5] G. Niu et al. Knowledge graph embeddings: a comprehensive survey on capturing relation properties. *arXiv preprint* 2410.14733, 2025.